ITM SKILLS UNIVERSITY

School of Future Tech

JAVA PROGRAMMING

Final Project Report

Case Study 34

BLOOD DONATION CAMP MANAGEMENT SYSTEM

Developed in Java (OOP + Collections + Swing GUI)

Detail	Information
Submitted by	Bhaumik Keer
Roll Number	150096725030
Programme	B.Tech CSE (2025-29), Semester III
Subject	Java Programming

Contents

1. Introduction and Objectives
2. Java Concepts Used (Case Study Mapping)
3. System Design
4. Validation, Exceptions, Searching and Sorting
5. Test Cases
6. Screens of the Application
7. Conclusion and Future Scope

How to run: install JDK 11 or newer, open Terminal in the BloodDonationCamp folder, then run javac *.java and java MainGUI. The complete source code (8 Java files) is submitted in the folder 1_Source_Code and is not repeated in this report.

1. Introduction and Objectives

Blood donation camps need careful record keeping: who has registered as a donor, which camp a donor will attend, how many units of blood were collected and how often a person has donated. On paper this is slow and easy to lose. This project is a Java desktop application that keeps these records and helps staff track donor participation and blood collection.

Problem statement (Case Study 34): A healthcare organization conducts blood donation camps and requires a system to manage donors, donation appointments, blood groups, donation records, and certificates.

The application has a simple Swing interface with six tabs. All data is stored in Java collections and every input is validated; wrong input is reported with a custom exception. Technology used: Java (JDK 11+, tested on JDK 21), Swing, java.time.LocalDate; runs on macOS, Windows and Linux.

Objectives and how they are met

Objective	How it is achieved	Tab
Register blood donors	Validated form; ID D001, D002... created automatically	Donors
Manage donation appointments	Book, change camp, delete; checks donor, camp, date, 90-day gap	Appointments
Record blood donations	Saved with today's date; donor and blood stock updated	Donations
Search donors by blood group	Linear search; search by ID uses a HashMap	Search & Sort
Maintain donation history	LinkedList, newest first; filter by donor	Donations
Donor reports and certificates	Reports and certificate shown on screen and saved as text files	Reports

2. Java Concepts Used (Case Study Mapping)

Java Concept	Application (as per case study)	Where in the code
Classes & Objects	Donor, Donation, Camp, Appointment	Donor.java, Donation.java, Camp.java, Appointment.java
Constructors	Donor / donation / camp initialization	public Donor(id, name, age, ...) and the same in each data class
Array	Blood group information	Validator.BLOOD_GROUPS (String[8]), BloodBank.stock (int[8])
ArrayList	Donors and camps	BloodBank.donors, camps, appointments
LinkedList	Donation history	BloodBank.history (addFirst = newest first)
HashMap	Donor ID to Donor	BloodBank.donorMap; findDonor()

Java Concept	Application (as per case study)	Where in the code
TreeMap	Donors sorted by blood group / name	BloodBank.sortByBloodGroup()
CRUD	Donor / camp / appointment management	add, update, delete methods in BloodBank
Searching	Search donor by ID / blood group	findDonor() (HashMap), searchByGroup() (linear)
Sorting	Sort donors by name / donation date	sortByName(), sortByDonationDate() (lambda)
Swing	Donation camp GUI	MainGUI.java: JFrame, JTabbedPane, JTable, JButton, ...
Exception Handling	Invalid donation / appointment	InvalidDonationException; try/catch in every button
Validation	Blood group, donor age, contact	Validator.java

3. System Design

3.1 Architecture

Layer	Files and role
Presentation	MainGUI.java: screens, text boxes, buttons, messages. No business rules.
Business logic	BloodBank.java: all collections and operations (CRUD, search, sort, reports). Throws InvalidDonationException.
Validation	Validator.java: checks for name, age, blood group, phone number, dates.
Data classes	Donor, Camp, Appointment, Donation: private fields, constructor, getters and setters.

Example flow: the user clicks Add Donor, MainGUI calls bank.addDonor(), BloodBank calls the Validator, a Donor object is stored in the ArrayList and the HashMap, and the tables are refreshed. If a rule fails, an exception is thrown and its message is shown in a message box.

3.2 Modules

Module	Implementation
Donor Registration	Donors tab; addDonor()
Blood Group Management	Blood group array, stock array, group summary in report
Donation Camp Management	Camps tab; addCamp(), updateCamp(), deleteCamp()
Appointment Management	Appointments tab; bookAppointment(), updateAppointment(), deleteAppointment()

Module	Implementation
Donation Recording	Donations tab; recordDonation()
Donor Search / Donor Sorting	Search & Sort tab; HashMap, linear search, lambda sorts, TreeMap
Donation History	LinkedList history; historyOf(donorId)
Certificate Generation / Donation Report	Reports tab; certificate(), overallReport(), donorReport()

3.3 Data structures and rules

Structure	Variable	Why
Array	BLOOD_GROUPS, stock	Fixed 8 blood groups; index links a group to its stock count
ArrayList	donors, camps, appointments	Grows automatically; easy to loop and sort
LinkedList	history	addFirst() keeps the newest donation first
HashMap	donorMap	Finds a donor by ID without looping
TreeMap	in sortByBloodGroup()	Keys stay sorted automatically

Business rules: donor age 18 to 65; contact exactly 10 digits; valid blood group; at least 90 days between two donations; an appointment needs an existing donor and camp, a camp date not in the past and no duplicate booking; a certificate needs at least one recorded donation.

4. Validation, Exceptions, Searching and Sorting

All checks are in Validator.java. When a rule is broken the code throws InvalidDonationException; the GUI catches it with try/catch and shows the message, so wrong input never crashes the program.

Input	Rule	Message shown
Name	Letters, 2-30 characters	Name must have only letters (2 to 30 characters).
Age	18 to 65	Donor age must be between 18 and 65.
Blood group	One of 8 valid groups	Invalid blood group. Use A+, A-, B+, ...
Contact	Exactly 10 digits	Contact must be exactly 10 digits.
Donation gap	At least 90 days	Amit Verma donated only 0 days ago. The minimum gap is 90 days.
Appointment	Donor and camp must exist	Donor 'D999' does not exist.

```
if (age < 18 || age > 65) {      // inside Validator.checkDonor()
    throw new InvalidDonationException("Donor age must be between 18 and 65.");
}
```

Operation	Technique	Complexity
Search by ID	HashMap lookup: donorMap.get(id)	O(1)
Search by blood group	Linear search: check every donor, collect matches	O(n)
Sort by name	ArrayList.sort() with a lambda comparing names	O(n log n)
Sort by donation date	Lambda on the date text (yyyy-mm-dd); latest first, never-donated last	O(n log n)
Sort by blood group	TreeMap with key "blood group + name + ID"; keys sorted automatically	O(n log n)

5. Test Cases

#	Test	Input	Result	Status
1	Valid donor	Age 21, B+, 10-digit phone	Donor added with new ID	Pass
2	Under-age donor	Age 15	Age must be between 18 and 65	Pass
3	Bad phone number	12345	Contact must be exactly 10 digits	Pass
4	Invalid blood group	Z+	Invalid blood group message	Pass
5	Donation too soon	Same donor twice, same day	Minimum gap is 90 days	Pass
6	Unknown donor	D999	Donor does not exist	Pass
7	Past camp / duplicate booking	C001 / D003 at C002 twice	Date passed / already booked	Pass
8	Search O+ and ID D005	Search tab	Matching donor(s) shown	Pass
9	Sort name / date / TreeMap	Buttons	Correct order shown	Pass
10	Certificate without donation	Donor D004	Cannot give certificate	Pass

6. Screens of the Application

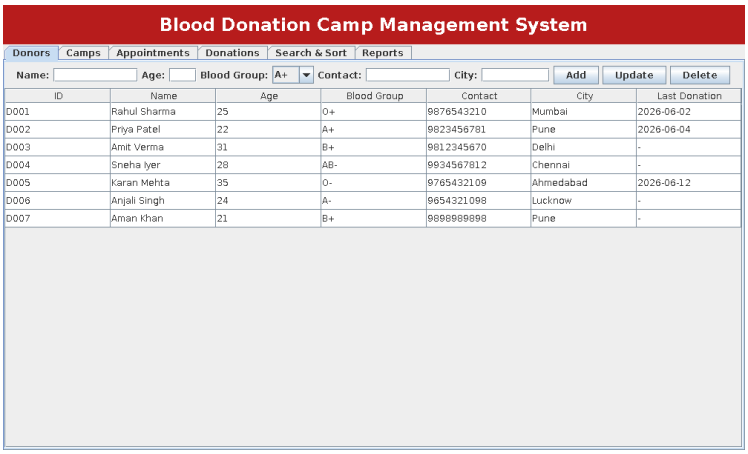


Figure 1: Donors tab - registration form and donor table

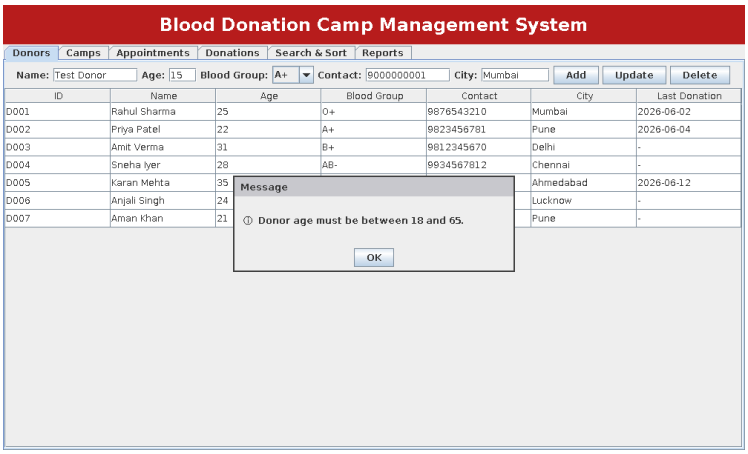


Figure 2: Validation error message (age below 18)

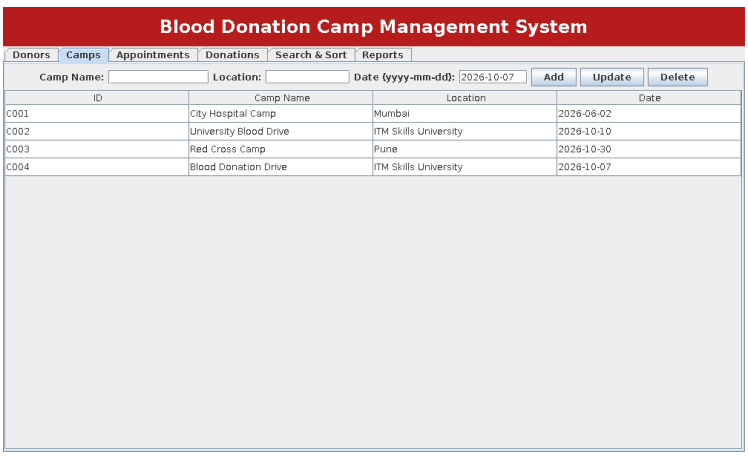


Figure 3: Camps tab

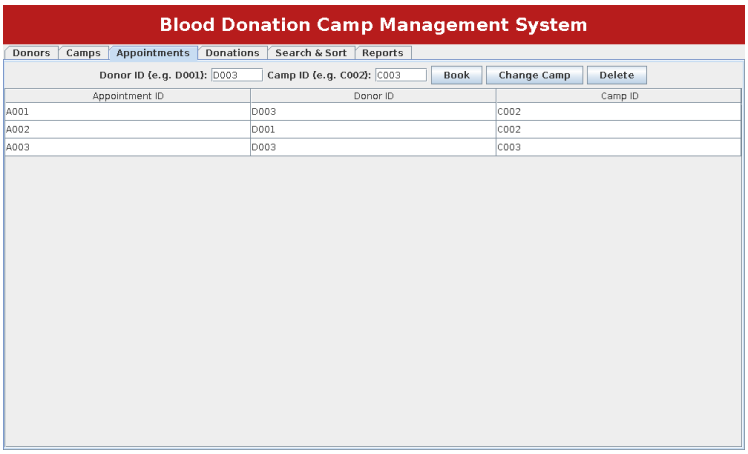


Figure 4: Appointments tab

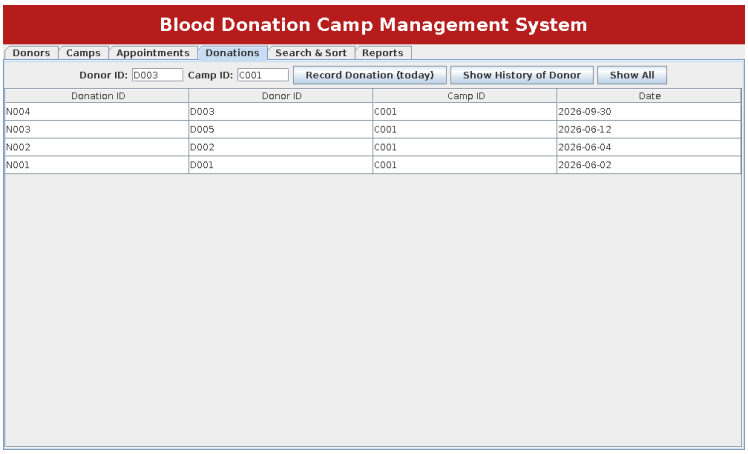


Figure 5: Donations tab with donation history

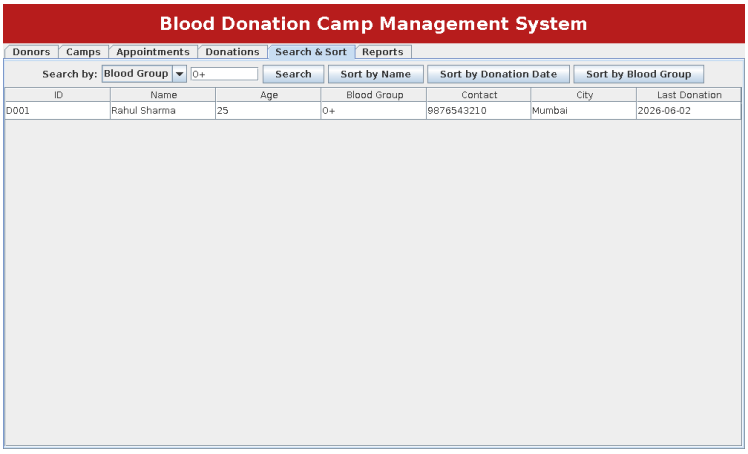


Figure 6: Search by blood group (O+)

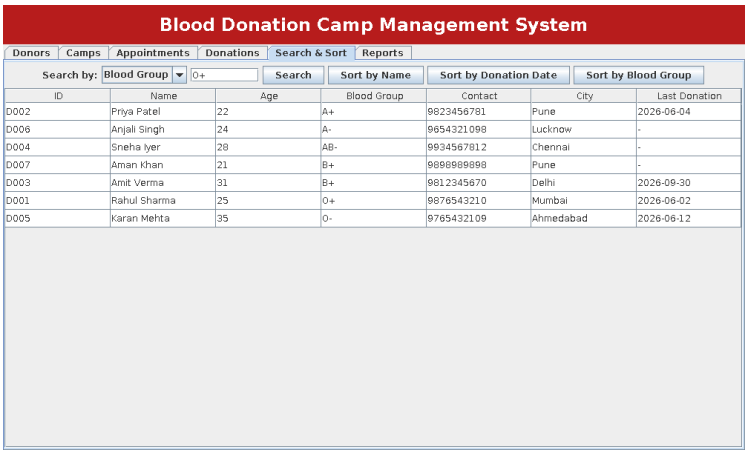


Figure 7: Donors sorted by blood group using TreeMap

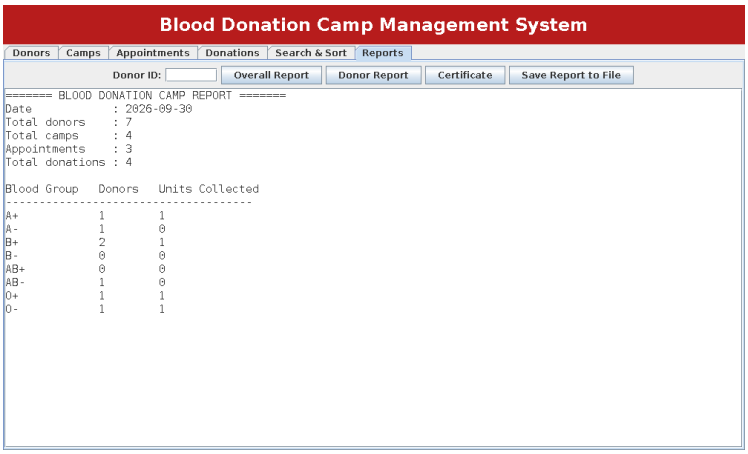


Figure 8: Overall report

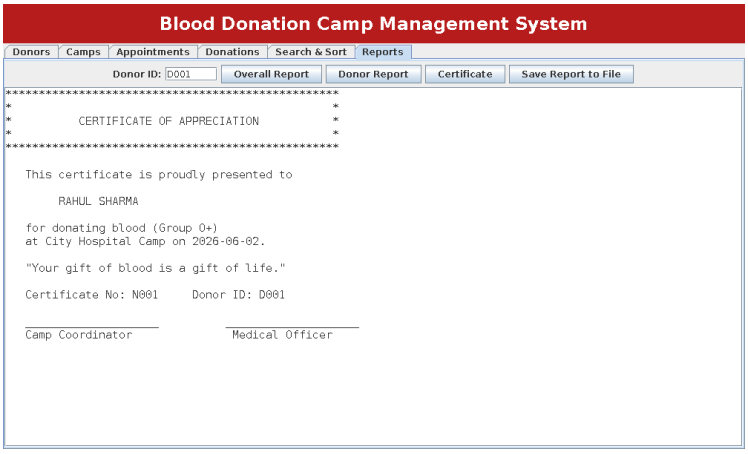


Figure 9: Donor certificate

7. Conclusion and Future Scope

The system meets every objective of Case Study 34 and shows how Java OOP, collections, searching and sorting, validation, exception handling and Swing work together in a practical healthcare application. Future scope: permanent storage in a file or database (JDBC), staff login, SMS or e-mail reminders and PDF reports.